

FSLE Python Script User Guide

Sara Cloux^{*1}, Cristobal López¹, Emilio Hernández¹

¹ *IFISC-UIB* (Institute for Cross-Disciplinary Physics and Complex Systems)



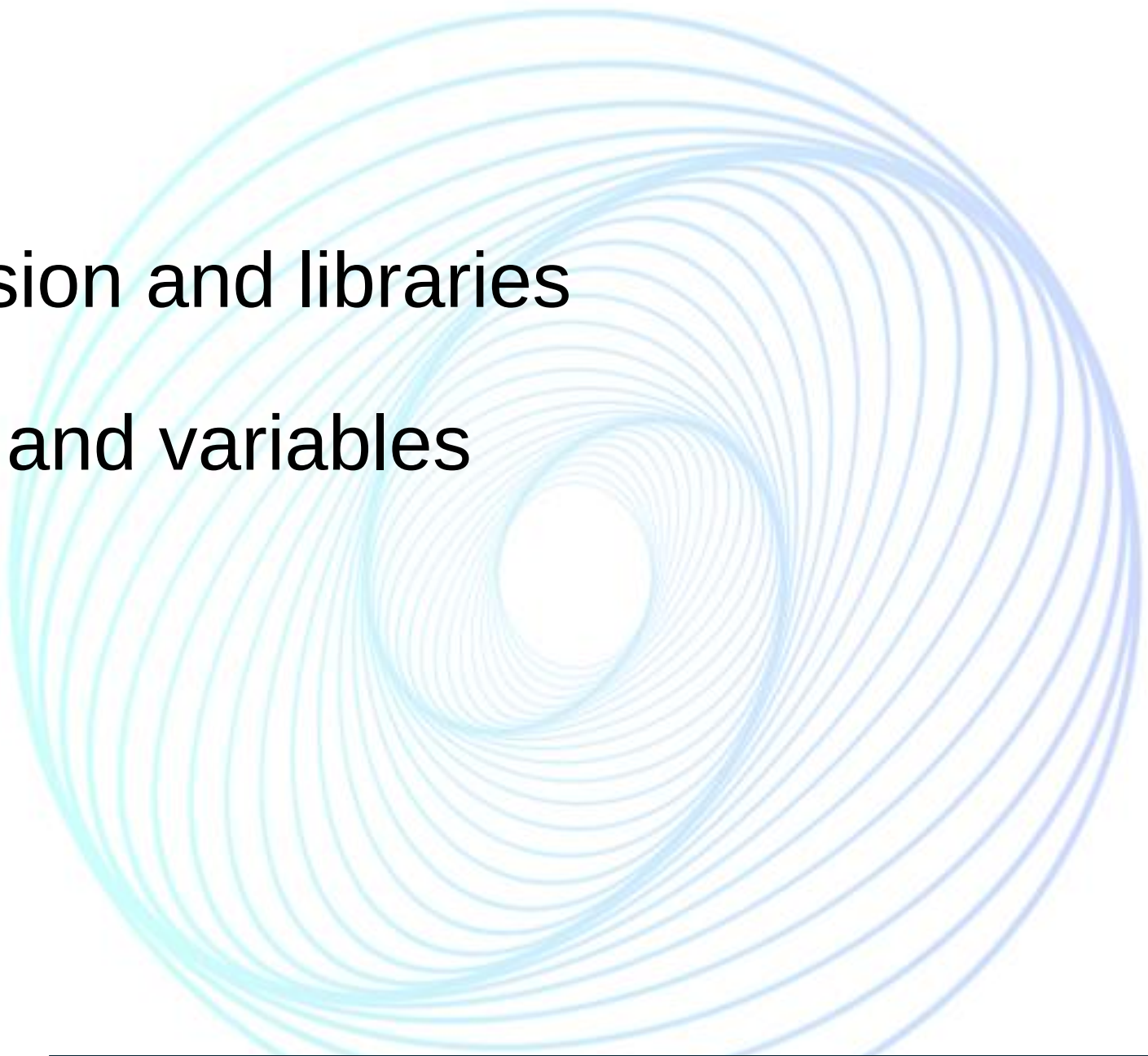
LAMARCA

Lagrangian transport of MARine litter and microplastics
from modeling, analysis and observations in Coastal waters



FSLE Python Script User Guide

Index



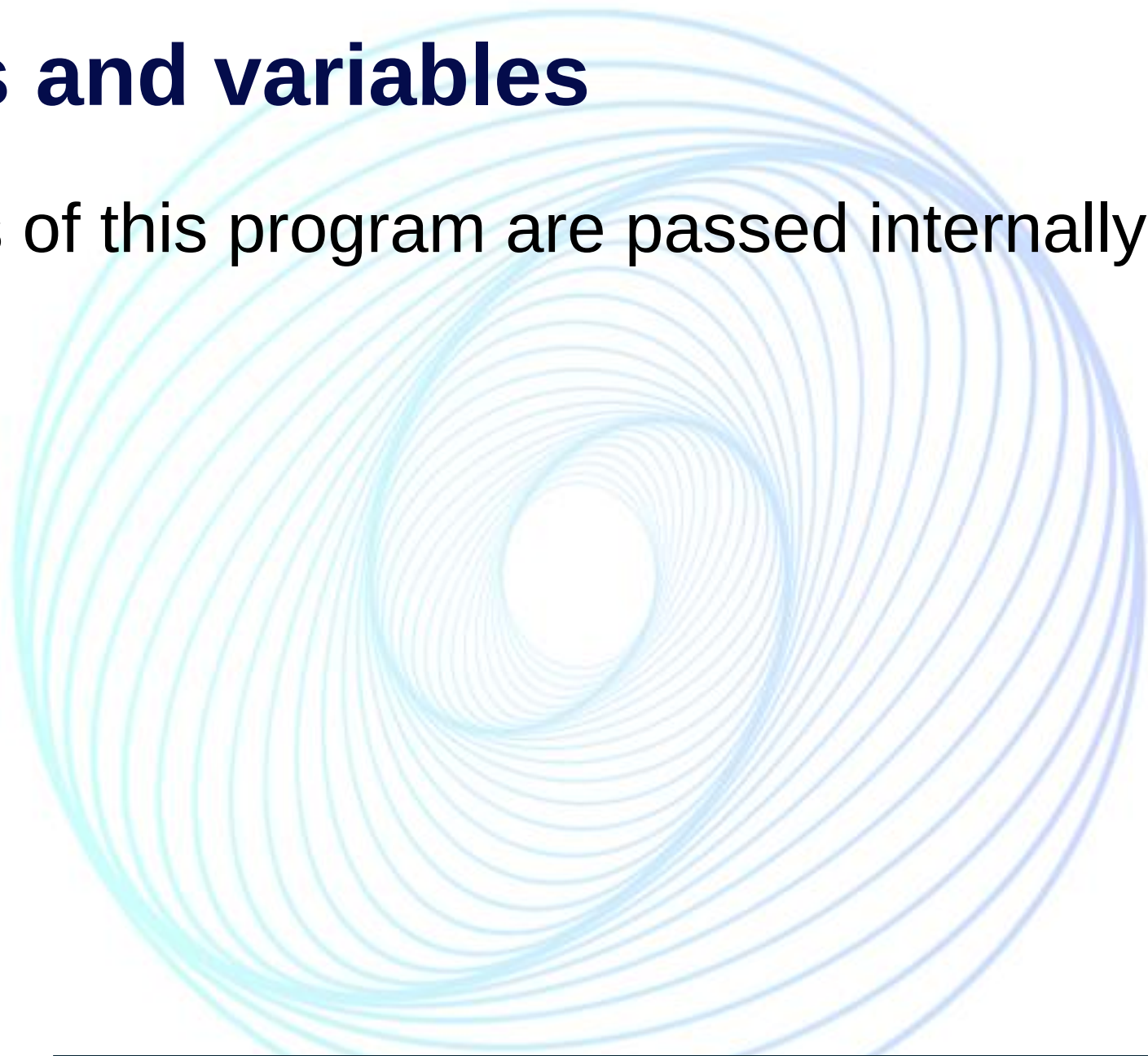
- . Python version and libraries
- . Arguments and variables
- . Functions

Python version and libraries

We use Python version 3.9, and the same libraries as in the 3D Particle Tracking Python Model. (see that User guide)

Arguments and variables

All arguments of this program are passed internally.



Arguments and variables

We load any trajectory file in order to extract the constants corresponding to the time step (dt_horas), the spatial resolution (dx), the depth (zeta) and the trajectories id (ids).

```
pwd_files = '/data/geo/lamarca/ROMS500/RESULTADOS/3D_15Days/backwards/depth_25/'
day_6 = 'BACK_2011-02-26T03:30:00_dx_0.0025_depht_-25_dt_1.nc'

ds = xr.open_dataset(pwd_files + day_6)
ds.load()

umbral = 0.05
dt_horas = 1.0
ids = np.transpose(np.reshape(ds.id.values, ds.reshape_dimensions))
dx=0.0025
zeta = day_6[-11:-8]
zeta = zeta.replace('_', '')
```

Arguments and variables

To calculate FSLE, we need to know the distance of each particle between its four nearest neighbors at each instant of time. To optimize the program, we initially obtain the id of the four neighbors with respect to the i-particle.

Arguments and variables

```
# Definir los desplazamientos de los vecinos
displacements = [(0, 1), (0, -1), (1, 0), (-1, 0)]

# Crear un diccionario para almacenar los vecinos de cada ID
neighbors = {}

# Buscar vecinos para cada punto en la matriz
for i in range(rows):
    for j in range(cols):
        id = ids[i, j]
        current_coords = (i, j)
        current_neighbors = []

        # Buscar vecinos usando los desplazamientos definidos
        for deltax, deltay in displacements:
            neighbor_coords = (i + deltax, j + deltay)
            if 0 <= neighbor_coords[0] < rows and 0 <= neighbor_coords[1] < cols:
                neighbor_id = ids[neighbor_coords]
                current_neighbors.append(neighbor_id)

        # Almacenar la tupla de vecinos en el diccionario
        neighbors[id] = tuple(current_neighbors)
```


Arguments and variables

```
filtered_neighbors = {key: value for key, value in neighbors.items() if len(value) == 4}
neighbors = [(clave,) + valores for clave, valores in filtered_neighbors.items()]
pepe = pd.DataFrame(neighbors)
```

Thus, pepe contains N rows (as many as N particles surrounded by 4 neighbors), where the first column corresponds to the id of the i-particle, and the second, third and fourth columns to the id of the particles above, below, to the right and to the left respectively.

Functions

There are 3 key functions in trajectory calculation: `spherical_distance`, `calcula_fsle` and `plot_and_save_fsle`.

The function `spherical_distance` calculates the spherical distance of two points expressed in latitude and longitude.

Functions

```
def spherical_distance(lat1, lon1, lat2, lon2):
    """
    Calcula la distancia esférica entre dos puntos en la Tierra dados
    sus latitudes y longitudes en grados.

    Args:
    lat1, lon1: Latitud y longitud del primer punto (en grados)
    lat2, lon2: Latitud y longitud del segundo punto (en grados)

    Returns:
    Distancia esférica entre los dos puntos en grados.
    """
    DEG2RAD = np.pi / 180.0
    lat1_rad = lat1 * DEG2RAD
    lon1_rad = lon1 * DEG2RAD
    lat2_rad = lat2 * DEG2RAD
    lon2_rad = lon2 * DEG2RAD

    xinterm = (np.cos(lat1_rad) * np.cos(lat2_rad) * np.cos((lon2_rad - lon1_rad)) +
               np.sin(lat1_rad) * np.sin(lat2_rad))
    if xinterm.any() > 1.0: print('quepasa?9')
    elif xinterm.any() < -1.0: print('quepasa?9')

    dist = np.arccos(xinterm) * (1.0 / DEG2RAD)

    return dist
```

Functions

The function `calcula_fsle` calculates in a vectorized way the value of fsle for each particle (for each row of the pepe dataset).

Functions

```
def calcula_fsle(ds):

    # ds['time'] = ds.time.values.astype('float64') / 1e9
    # t_matrix = np.zeros(ds.reshape_dimensions)
    # dis_matrix = np.zeros(ds.reshape_dimensions)

    tuplas_resultado=([])
    primeros_valores=([])
    for t in range(len(ds.time)):
        dist_d = spherical_distance(ds.latitude[pepe[1],t].values, ds.longitude[pepe[1],t].values, ds.latitude[pepe[0],t].values, ds.longitude[pepe[0],t].values)
        dist_u = spherical_distance(ds.latitude[pepe[2],t].values, ds.longitude[pepe[2],t].values, ds.latitude[pepe[0],t].values, ds.longitude[pepe[0],t].values)
        dist_r = spherical_distance(ds.latitude[pepe[3],t].values, ds.longitude[pepe[3],t].values, ds.latitude[pepe[0],t].values, ds.longitude[pepe[0],t].values)
        dist_l = spherical_distance(ds.latitude[pepe[4],t].values, ds.longitude[pepe[4],t].values, ds.latitude[pepe[0],t].values, ds.longitude[pepe[0],t].values)

        df_dist_d = pd.DataFrame(dist_d, columns=['0'])

        df_dist_u = pd.DataFrame(dist_u, columns=['1'])

        df_dist_r = pd.DataFrame(dist_r, columns=['2'])

        df_dist_l = pd.DataFrame(dist_l, columns=['3'])

        result = pd.concat([df_dist_d, df_dist_u, df_dist_r, df_dist_l], axis=1)
        result.index = pepe[0].values
        result = result.drop(index = primeros_valores, errors='ignore')
        result['max_value'] = result.max(axis=1, skipna=True)

        filtered_df = result[result['max_value'] >= umbral]
        # index_to_drop.append(filtered_df.index.values)
        # index_to_drop = np.unique(index_to_drop)

        for idx, row in filtered_df.iterrows():

            tuplas_resultado.append((idx, row['max_value'], t))
            print(idx, row['max_value'], t)
    primeros_valores = [tupla[0] for tupla in tuplas_resultado]

    return tuplas_resultado
```

Functions

The function `plot_and_save_fsle` plots a figure with the python `imshow` function to show the FSLE. It also saves the image, in our case, in png format.

Functions

```
def plot_and_save_fsle(tuplas):

    ratio = umbral/dx
    ides, dist, time_stamp = ([tupla[i] for tupla in tuplas] for i in range(3))

    tau = [ts * (dt_horas / 24.0) for ts in time_stamp]
    log_dis = np.log(ratio)
    fsle_final = [log_dis / t for t in tau]
    df_fsle = pd.DataFrame(fsle_final, columns=['FSLE'])
    df_fsle.index = ides

    # Crear una nueva matriz llena de np.nan con las mismas dimensiones que ids
    fsle_matrix = np.transpose(np.full(ds.reshape_dimensions, np.nan))

    indices = df_fsle.index.values
    fsle_values = df_fsle['FSLE'].values

    # Rellenar fsle_matrix con los valores de FSLE correspondientes
    for idx, value in zip(indices, fsle_values):
        # Encontrar la posición del índice en la matriz ids
        position = np.where(ids == idx)
        if position[0].size > 0 and position[1].size > 0: # Asegurar que se encuentra el índice
            fsle_matrix[position] = value

    lat_dimension = ds.latitude_init[0:ds.reshape_dimensions[1]]
    lon_dimension = ds.longitude_init[0::ds.reshape_dimensions[1]]

    plt.figure(figsize=(10, 8))
    plt.imshow(((np.flipud(fsle_matrix[:,-2]))),
               vmin=0,
               vmax=14,
               extent=[lon_dimension[1], lon_dimension[-2], lat_dimension[1], lat_dimension[-2]],
               cmap='binary')

    cbar = plt.colorbar(orientation='horizontal', pad=0.1)
    cbar.set_label('Days$^{ -1}$ [s]')
    plt.xlabel('Longitude [°]')
    plt.ylabel('Latitude [°]')
    plt.title(r'$\Delta_f = {} \Delta_0$ ; {}'.format(ratio, file_name[5:-31]), fontsize=16)

    os.makedirs(folder_path, exist_ok=True)
    output_path = os.path.join(folder_path, 'fsle_matrix_ratio_'+str(ratio)+'_'+file_name[: -3]+' .png')
    plt.savefig(output_path, dpi=300) # Guardar con alta resolución (300 dpi)
```